

RAG Learning Guide

A Complete Guide to Retrieval-Augmented Generation
from Scratch using Python & Gemini API

Prepared for: AI Automation Engineering Learning

Created at: C:/Users/anand/rag_bot/docs/

What is this Project? (RAG Demystified)

This project is a Retrieval-Augmented Generation (RAG) application. It is one of the most important concepts to learn as an AI Automation Engineer.

The Problem RAG Solves: "Hallucinations"

Large Language Models (LLMs) like GPT, Claude, or Gemini are trained on public internet data up to a certain point in time.

If you ask an LLM:

- "What is my personal schedule for tomorrow?"
- "What are the terms of this custom contract my company just signed?"

The LLM cannot answer because it does not have access to your private files. If it tries to answer, it might make up convincing-sounding but completely false facts. This is called a hallucination.

The Solution: Retrieval-Augmented Generation

Instead of retraining the LLM (which is very expensive and slow), RAG works by giving the LLM the exact page/context it needs to answer the question right before it starts typing.

Think of it like an open-book exam:

1. You ask a question.
 2. The system searches your private documents ("the textbook") for the most relevant paragraphs.
 3. It copies those paragraphs and pastes them into a prompt template.
 4. It hands this prompt to the LLM (Gemini) and says: "Answer the user's question, but use ONLY these paragraphs."
 5. The LLM reads the paragraphs and writes a 100% accurate, fact-checked answer.
-

The Two Phases of the Project

Phase 1: Ingestion (ingest.py)

This is the "study phase" where documents are prepared.

1. Extraction: Reading the text from files (like PDF or TXT).
2. Chunking: Cutting long text into small, readable chunks (700 characters each).
3. Embedding: Converting text chunks into mathematical representation vectors.
4. Storage: Saving these vectors and chunks to our local database (vector_db.json).

Phase 2: Q&A / Chat (query.py)

This is the "interactive phase" where the bot answers questions.

1. Query Embedding: Converts your question into a mathematical vector.
2. Similarity Search: Looks through vector_db.json and calculates the Cosine Similarity between your query vector and all document chunk vectors. It grabs the top 2 closest matching chunks.
3. LLM Context Generation: Passes the matching chunks to gemini-2.5-flash along with your question.
4. Answer: The LLM writes the final grounded response.

Technologies & Concepts Used in this Project

Here are the details of the tools and engineering concepts utilized in this RAG codebase.

1. Vector Embeddings (gemini-embedding-001)

An embedding model converts a string of text into a vector (a list of 768 decimal numbers).

- Why is this useful? Words that mean similar things are closer to each other in vector space. For example, if you query "How do I solve bugs?", the model can match it to a document chunk that says "Debugging error reports" even though the word "bugs" is not in that chunk! This is called Semantic Search.

- Task Types:
 - retrieval_document: Used when embedding the document chunks to optimize them for database storage.
 - retrieval_query: Used when embedding the user's question to optimize it for searching.
-

2. Cosine Similarity Math

How do we find which vectors in our database match our query vector? We measure the angle between them using Cosine Similarity.

- If two vectors point in the exact same direction (very similar meaning), the cosine similarity score is 1.0.
 - If they are perpendicular (unrelated), the score is 0.0.
 - Our custom vector_db.py calculates this using a simple loop over the vector arrays.
-

3. Large Language Model (gemini-2.5-flash)

We use Gemini 2.5 Flash for the final answer generation.

- Why Flash? It is extremely fast, highly cost-effective, and supports a massive context window (up to 1 million tokens).
 - Prompting constraints: In query.py, we construct a prompt instructing the LLM: "Use ONLY the facts provided in the Context section. If the answer cannot be found, state that you do not have enough information." This forces the LLM to remain grounded.
-

4. Local File-Based Storage (vector_db.json)

Instead of installing an enterprise database engine (like Pinecone or PostgreSQL pgvector), we build a local file-based database.

- How it works: When chunks are processed, the Python lists and floats are dumped into a standard JSON file.
- Benefit: You can open vector_db.json in any text editor and see exactly what vectors look like. It is 100% transparent

and requires zero setup/installation.

5. Chunking & Overlap

When reading `sample_doc.txt`, we split it into segments of 700 characters with an overlap of 150 characters.

- Why chunk size is 700: If chunks are too small, they lose context. If they are too big, they contain too much noise and waste API tokens.
- Why overlap is 150: Overlapping ensures that a sentence split right in the middle during chunking isn't lost. The meaning is carried over to the next chunk.

How the Code Works: Step-by-Step

This guide walks through how the Python files interact to power your RAG application.

Code Workflow Map

Script Descriptions

1. vector_db.py (The Database Manager)

This file handles the storage.

- class SimpleVectorDB: Holds database records in memory.
 - self.load() / self.save(): Reads from and writes to the local vector_db.json file.
 - self.cosine_similarity(v1, v2): Takes two lists of numbers, multiplies matching indexes together, and calculates the angle. This tells us how similar the two vectors are.
 - self.query(): Loops through all items in the database, runs cosine similarity on each, sorts them, and returns the best matches.
-

2. ingest.py (The Document Processor)

This script loads and processes documents to populate your database.

- extract_text_from_file(file_path): Reads .txt files directly. For .pdf files, it uses the pypdf library to loop through pages and extract raw text.
 - chunk_text(text): Splits the extracted text into chunks.
 - embed_text(chunk): Calls genai.embed_content() using the model models/gemini-embedding-001 to fetch the 768-number vector for that chunk.
 - db.add(): Calls our custom database script to save the text chunk, the vector list, and the document name.
-

3. query.py (The Q&A CLI Interface)

This script runs the interactive loop in your terminal.

- Checks database: Verifies that vector_db.json is loaded.
- embed_query(question): Converts your typed question into a vector using the same embedding model.
- db.query(): Fetches the top 2 matching text chunks from the database. It prints out the exact similarity score so you can inspect how well they matched!

- Generates response: Feeds the matching context chunks, the user's question, and a strict prompt instructing Gemini to only use the provided context to answer the question.